Development & Integration

Date	13 July 2026
Team ID	
Project Name	Voice-Based Concept Understanding Analyzer (VBCUA)

Development & Integration

This section describes how the Voice-Based Concept Understanding Analyzer (VBCUA) was developed and integrated end-to-end, covering the frontend, backend, code structure, error handling, and security aspects of the implementation.

Development & Integration	**Frontend development (Streamlit / Flask / Web UI):** The application's frontend is built entirely with Streamlit, organised into dedicated UI components: the upload section (upload_section.py), results panel (results_panel.py), report download actions (report_actions.py), and sidebar (sidebar.py). The interface is configured with a wide layout, a custom colour theme (.streamlit/config.toml), and minor inline CSS adjustments in main.py. **Backend logic implementation:** Backend processing is implemented as a set of dedicated Python modules under app/modules: transcriber.py (speech-to-text via Whisper), semantic_analyzer.py (embedding generation and cosine similarity via Sentence Transformers), concept_scorer.py (score-to-grade conversion and feedback generation), and report_generator.py (PDF report creation via ReportLab). main.py orchestrates the end-to-end flow between these modules and the UI. **Gemini API integration:** ***[Not Applicable]*** VBCUA does not integrate with the Gemini API or any other external generative AI API. Both models used — Whisper and the Sentence Transformer — run locally within the application process via the openai-whisper and sentence-transformers Python packages, with no outbound calls to a third-party AI service. **Database integration:** ***[Not Applicable]*** No database (relational or NoSQL) is integrated in the current version. Persistent data — uploaded audio, the reference answer, and generated PDF reports — is stored directly on the local filesystem (data/audio, data/reference_answers, reports/), and transient state during a session is held in Streamlit's session_state. Future enhancement: introducing a database would be necessary to support features such as multi-user score history, which is currently identified as future scope. **Modular code structure:** The codebase is organised into clearly separated packages: app/modules for processing logic, app/ui for Streamlit UI components, and app/utils for shared helpers (file_io.py for reading the reference answer, with validators.py, paths.py, and logger.py present as placeholders for future utility functions). This separation keeps each processing stage independently testable and maintainable. **Error handling:** Each backend module defines and raises specific, descriptive exceptions rather than allowing unhandled errors to crash the application: TranscriptionError (transcriber.py), SemanticAnalysisError (semantic_analyzer.py), and standard ValueError / FileNotFoundError for invalid or missing inputs in concept_scorer.py, report_generator.py, and file_io.py. main.py wraps calls to these modules in try/except

	blocks and surfaces user-friendly warnings or errors through the Streamlit interface instead of a raw stack trace. **API key security:** ***[Not Applicable]*** No API key is required or used in the current version, since no external API is called. An empty .env.example file is present in the repository as a placeholder for environment-based configuration, but no secrets are currently loaded or required for the application to run. Future enhancement: should an external API (e.g., a cloud-based model or service) be integrated later, credentials should be loaded from environment variables via python-dotenv (already listed as a dependency) rather than hard-coded, and .env should remain excluded from version control (it is already listed in .gitignore).